

TP : Dump et Analyse d'un Processus

Objectif : Capturer et analyser la mémoire d'un processus pour extraire des données sensibles

Installation

```
# Installer les outils nécessaires
sudo apt-get update
sudo apt-get install gdb binutils
```

Démarrage Rapide

1. Compiler et lancer le programme

```
make
./vulnerable_app
```

Le programme affiche son PID - notez-le !

2. Dans le programme, explorez le menu :

- **Option 5** : Créer des données sensibles en mémoire
- **Option 6** : Afficher les données (pour vérifier)
- **Option 7** : Mode attente (pour faire le dump)

3. Dans un autre terminal, créer le dump :

```
sudo gcore -o dump PID
# Exemple : sudo gcore -o dump 12345
```

4. Analyser le dump :

```
# Méthode rapide
strings dump.* | grep -i "password"
strings dump.* | grep -i "api_key"
strings dump.* | grep -i "mysql"
```

Détails du TP

Menu du programme

- **1** : Lister les utilisateurs (4 pré-chargés)
- **2** : Ajouter un utilisateur
- **3** : Test buffer overflow
- **4** : Simuler activité réseau
- **5** : Créer données sensibles (heap)
- **6** : Afficher toutes les données
- **7** : Mode attente pour dump ★

Analyse du Dump

Recherche manuelle avec strings

Exemple :

```
# Rechercher les credentials
strings dump.* | grep -i "password"
strings dump.* | grep -i "admin"

# Rechercher l'API key

# Rechercher tokens et secrets

# Database

# Emails

# Cartes bancaires
```

Étape 3 : Analyse Initiale

3.2 Analyse avec objdump et readelf

```
# Informations sur le format du dump
file dump.*

# Examiner les sections (si ELF)
readelf -h dump.*

# Désassembler des sections spécifiques (avancé)
objdump -d dump.* | less
```

EXERCICE 3.1

Recherchez les informations sensibles dans le dump (Exemple) :

```
# Mots de passe
strings dump.* | grep -i "pass" > potential_passwords.txt
cat potential_passwords.txt

# Emails

# Clés API / Tokens

# Database credentials

# Cartes bancaires
```

Questions :

- Combien d'utilisateurs trouvés ? _____
- Combien d'emails trouvés ? _____
- Avez-vous trouvé l'API key ? _____
- Avez-vous trouvé le mot de passe de la DB ? _____

Désassembler des sections spécifiques

Étape 4 : Extraction de Données Sensibles

4.1 Recherche avec hexdump

```
# Recherche de patterns hexadécimaux
hexdump -C dump.* | grep -i "admin" > dump_secrets_admin.txt
hexdump -C dump.* | grep -i "secret" > dump_secrets.txt

# Recherche d'en-têtes HTTP (si activité réseau simulée)
hexdump -C dump.* | grep -i "HTTP" > http_headers.txt

# Afficher en hex + ASCII
hexdump -C dump.* | less
```

4.2 Extraction avancée avec grep

```
# Rechercher des patterns spécifiques
strings dump.* | grep -iE "(token|secret|key)" > sensitive_keywords.txt

# Cookies et sessions
```

```
strings dump.* | grep -i "session" > session_data.txt
```

- ✉ Adresses emails
- 🏠 Numéros de cartes bancaires
- 🌐 Adresses IP
- 🔗 URLs
- 🔑 Clés API
- 🔒 Mots de passe
- 🪪 Tokens et JWT
- ☁ Clés AWS
- 🔑 Clés privées SSH/RSA
- 🍪 Cookies de session

📊 Étape 5 : Rapport et IOCs

5.1 Créer un rapport structuré

Créez un fichier `RAPPORT\_ANALYSE.md` contenant :

```markdown

# Rapport d'Analyse - Dump Processus

## 1. Informations Générales

- \*\*Date de capture :\*\*
- \*\*Processus ciblé :\*\*
- \*\*PID :\*\*
- \*\*Taille du dump :\*\*
- \*\*Système d'exploitation :\*\*

## 2. Méthodologie

- \*\*Outils utilisés :\*\*
  - gcore
  - strings
  - volatility3
  - scripts custom

## 3. Découvertes

### 3.1 URLs visitées

[Lister les URLs intéressantes]

### 3.2 Credentials trouvés

[ATTENTION : données sensibles]

### 3.3 Connexions réseau

[IPs, ports, protocoles]

### 3.4 Données sensibles

[Emails, tokens, cookies, etc.]

## 4. Indicators of Compromise (IOCs)

### ### IP Addresses

192.168.1.100 203.0.113.45

### ### URLs suspectes

http://malicious-site.com

### ### Hashes de fichiers

MD5: xxxxx SHA256: yyyyy

## ## 5. Recommandations

- [ ] Changer les mots de passe exposés
- [ ] Révoquer les tokens d'authentification
- [ ] Analyser les connexions suspectes
- [ ] Implémenter le chiffrement mémoire

## ## 6. Conclusion

[Résumé des findings]

---

## ## 🚀 Exercices Avancés (Bonus)

### ### Challenge 1 : Buffer Overflow

1. Dans le programme, choisissez l'option **\*\*3\*\*** (Test d'entrée vulnérable)
2. Essayez d'entrer une longue chaîne (> 64 caractères)
3. Faites un dump après le crash (si le programme plante)
4. Analysez la stack pour voir le dépassement de buffer

```
```bash
```

```
# Exemple d'entrée longue
```

```
python3 -c "print('A' * 100)"
```

Challenge 2 : Analyse avec GDB

1. Attachez GDB au processus en cours
2. Examinez la mémoire des structures de données
3. Lisez directement les passwords en mémoire

```
# Lancer avec GDB
gdb ./vulnerable_app

# Ou attacher à un processus en cours
gdb -p PID

# Dans GDB
(gdb) info proc mappings
(gdb) x/100s 0xADDRESSE # Examiner la mémoire
(gdb) find 0xADDRESSE_DEBUT, 0xADDRESSE_FIN, "password"
```

Challenge 3 : Memory Leak

1. Le programme ne libère PAS la mémoire allouée (option 5)
2. Utilisez **valgrind** pour détecter les fuites
3. Identifiez où se trouvent les données non libérées

```
# Analyser les fuites mémoire
valgrind --leak-check=full ./vulnerable_app

# Ou avec des options détaillées
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes
./vulnerable_app
```

Challenge 4 : Forensics avec hexdump

1. Utilisez **hexdump** pour examiner le dump en hexadécimal
2. Recherchez les patterns de la structure **User**
3. Trouvez l'offset exact des passwords

```
# Voir en hexadécimal + ASCII
hexdump -C dump.* | grep -i "password"

# Extraire une zone mémoire spécifique
hexdump -C dump.* | head -1000 > hex_analysis.txt
```

1. Établissez une connexion SSH
2. Dumppez le processus ssh
3. Recherchez des traces de la clé de session

Challenge 3 : Forensics avancé

1. Utilisez Volatility 3 pour une analyse complète
2. Identifiez les DLLs/bibliothèques chargées
3. Examinez les threads actifs

4. Recherchez des injections de code suspectes